सी डैक | प्रगत संगणन विकास केंद्र
CDAC | CENTRE FOR DEVELOPMENT OF ADVANCED COMPUTING

Machine Learning Algorithms in HPC

Introduction:

A decision tree is a tree like structure which mimics the process of decision-making by breaking down data into smaller subsets based on specific features.

Each internal node of the tree represents a decision based on a feature, and each leaf node represents the final prediction or decision.

Scikit-Learn uses the classification and regression tree (CART) algorithm to train decision trees. It produce only binary trees: non-leaf nodes always have two children (i.e questions only have yes/no answers).

Why decision tree was used in the research paper?

In this research paper, we used decision tree regression model because it effectively captured the complex, non-linear relationships between the input job parameters and the actual resource usage on HPC systems. The problem of predicting the required memory and runtime for HPC jobs involves many variables such as the account type, requested memory, number of nodes, and time limits. These variables do not have simple linear relationships with the job's actual performance.

Decision Trees do not assume any underlying relationship between the features and the target variable. Instead, they split the dataset into smaller and more homogenous subsets where the target values are more predictable. This characteristic made Decision Trees well-suited for modeling the unpredictable and varied nature of HPC job data.

The researchers did not train a single global model but instead used a Mixed Account Regression Model (MARM) approach. This meant that data was partitioned based on job accounts, and models were trained separately for each account or group of accounts. In these smaller, more homogeneous datasets, Decision Trees are highly effective. They are fast to train and less prone to

overfitting when appropriately limited in depth. Random Forests and LightGBM, while powerful on large, complex datasets, introduce additional computational overhead. For small account-specific subsets, this overhead did not justify the relatively small gain in prediction accuracy. HPC administrators and users could easily understand how the tree splits decisions based on input features like requested memory, time limits, and CPU counts.

How was decision tree used in this research paper ?

Decision Tree Regression (DTR) was used as one of the primary machine learning models to predict the required memory (MaxRSS) and execution time (CPUTimeRAW) for jobs submitted to Slurm-based HPC systems.

They used in the following way:

1) The researchers first preprocessed the historical job data from two major HPC clusters (RMACC Summit and Beocat) by extracting relevant features such as the job account, requested memory, requested CPUs, time limits, nodes, and quality of service (QOS).

2) After encoding categorical variables and normalizing numerical data, they applied several machine learning models, including Decision Trees, to build predictive models for estimating how much memory and CPU time a job would actually consume.

3) The Decision Tree model was implemented using the scikit-learn library and trained separately for different subsets of the data.

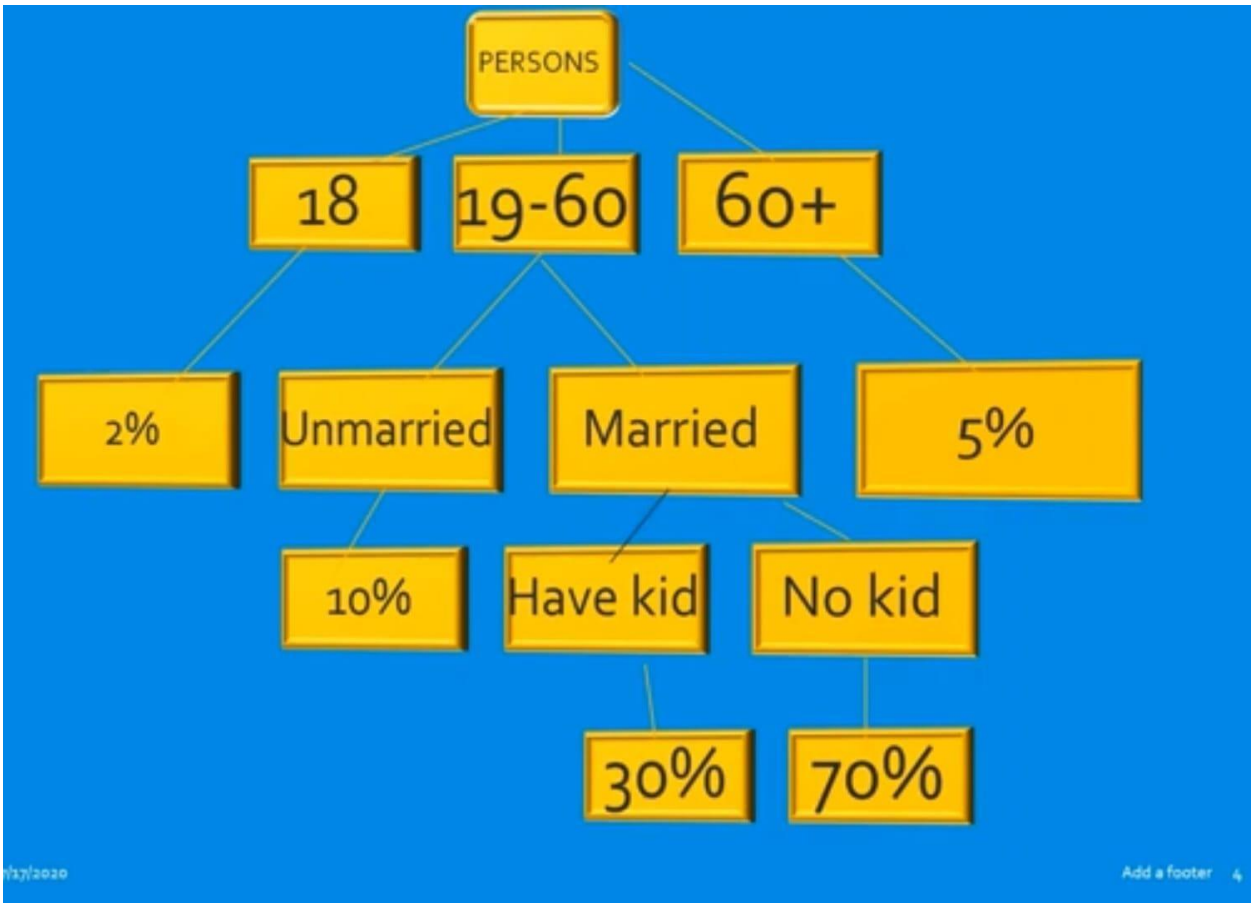
The authors used an innovative approach called the Mixed Account Regression Model (MARM), where they divided the data based on HPC user accounts. They trained a separate Decision Tree for each subset of accounts, optimizing the selection of accounts to maximize the predictive accuracy. This approach helped the model handle variability across different user workloads, as some accounts had more predictable job patterns. Decision Trees were particularly useful here because they can efficiently handle the non-linear relationships and varied usage patterns present in the HPC job data.

Decision Trees provided strong performance in terms of the R^2 score compared to other algorithms, showing their capability to explain a large portion of the variance in resource usage.

Decision Tree Regression

Decision Tree Regression is a supervised machine learning algorithm used to predict continuous numerical values. Unlike classification trees that predict discrete class labels, a regression tree predicts a continuous output. It works by learning decision rules from input data features and breaking down the dataset into smaller and smaller subsets, where each final subset corresponds to a predicted numeric value.

Example:



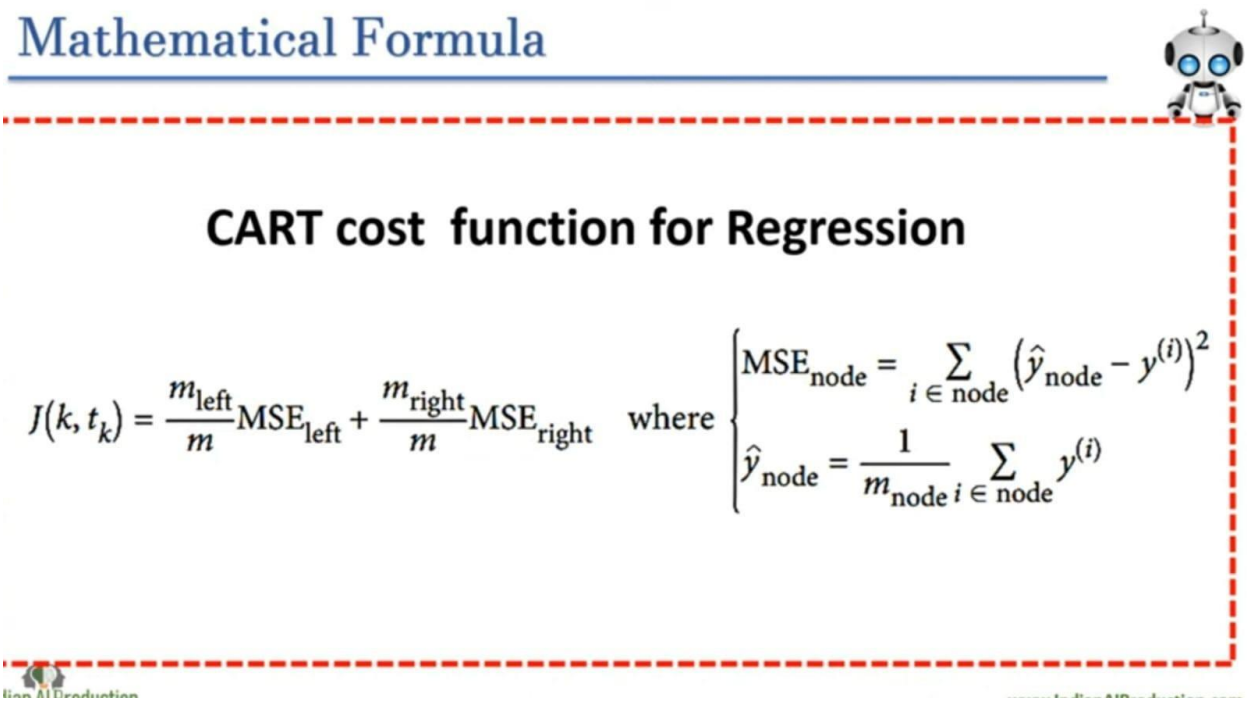
Working:

1) It starts with the entire dataset and looks for a feature and threshold value that split the data into two groups in a way that minimizes prediction error. The most

common metric used here is the Mean Squared Error (MSE) of the target values within each split.

2) Each of these groups is further split using the same logic, creating a tree-like structure.

3) When the tree cannot be split further, a leaf node is created. This leaf holds a single prediction value, which is the average of the target values in that subset of the data.



Why decision tree performed better?

1) HPC job data (features like Account, QOS, Timelimit, ReqNodes, etc.) is heterogeneous and non-linear.

2) Decision Trees can easily model non-linear relationships and split data into appropriate segments without assuming a linear pattern compared to other ML models.

3) Decision Trees can easily model non-linear relationships and split data into appropriate segments without assuming a linear pattern.

4) Decision Trees are fast to train.

5) They don’t overfit badly if pruned properly

6) They provide good performance on partitioned data.

7) Although Random Forest and LightGBM often outperform single Decision Trees on larger and complex datasets, random forest was slower to train on each small models. LightGBM performed well but, in some cases, didn't justify the additional complexity for the scale of each account's data slice.

Implementation:

```
import pandas as pd
import numpy as np
from sklearn.tree import DecisionTreeRegressor
from sklearn.model_selection import train_test_split
from sklearn.metrics import r2_score, mean_squared_error
import matplotlib.pyplot as plt
df = pd.read_csv("/content/dcgm.csv")
#Data preprocessing
df = df.drop(columns=["Node", "id_job"]) #Drop non numeric
columns
df = df[(df["totalexecutiontime_sec"] > 0) &
(df["maxgpumemoryused_bytes"] > 0)]
df = df.dropna()  #Drop rows with zero or missing execution
time
#Normalize power and memory columns to improve accuracy
from sklearn.preprocessing import MinMaxScaler
scaler = MinMaxScaler()
cols_to_normalize = ["powerusage_watts_avg",
"maxgpumemoryused_bytes"]
df[cols_to_normalize] =
scaler.fit_transform(df[cols_to_normalize])
# Derived feature: energy consumption proxy
df["power_time_product"] = df["powerusage_watts_avg"] *
df["totalexecutiontime_sec"]

# Derived feature: memory demand over time (with safety against
division by zero)
df["memory_per_sec"] = df["maxgpumemoryused_bytes"] /
df["totalexecutiontime_sec"].replace(0, np.nan)

# Optional: Drop rows with NaN (in case of divide-by-zero)
df.dropna(inplace=True)
X = df.drop(columns=["totalexecutiontime_sec",
"maxgpumemoryused_bytes"]).values
y_time = df["totalexecutiontime_sec"].values
df = df.replace([np.inf, -np.inf], np.nan).dropna()
X_train_time, X_test_time, y_train_time, y_test_time =
train_test_split(X, y_time, test_size=0.2, random_state=42)
#PART 1:- predicting CPUTimeRAW
time_model = DecisionTreeRegressor(random_state=42)
time_model.fit(X_train_time, y_train_time)
y_pred_time = time_model.predict(X_test_time)
```

```
print("Predicting CPUTimeRAW")
print("R2 Score:", r2_score(y_test_time, y_pred_time))
print("RMSE:", np.sqrt(mean_squared_error(y_test_time,
y_pred_time)))
plt.scatter(y_test_time, y_pred_time, color='green')
plt.plot([y_test_time.min(), y_test_time.max()],
[y_test_time.min(), y_test_time.max()], 'k--', lw=2)
plt.xlabel('Actual Time(sec)')
plt.ylabel('Predicted Time(sec)')
plt.title('Actual time vs Predicted time')
plt.show()

#Predictind MaxRSS i.e amount of physical memory required by
the job
X_mem = df.drop(columns=["totalexecutiontime_sec",
"maxgpumemoryused_bytes"]).values
y_mem = df["maxgpumemoryused_bytes"].values
X_train_mem, X_test_mem, y_train_mem, y_test_mem =
train_test_split(X, y_mem, test_size=0.2, random_state=42)
mem_model = DecisionTreeRegressor(random_state=42)
mem_model.fit(X_train_mem, y_train_mem)
#memory prediction
y_pred_mem = mem_model.predict(X_test_mem)
print("Predicting MaxRSS:")
print("R2 score:", r2_score(y_test_mem, y_pred_mem))
print("Root Mean Square Error", mean_squared_error(y_test_mem,
y_pred_mem))

plt.scatter(y_test_mem, y_pred_mem, color='blue')
plt.plot([y_test_mem.min(), y_test_mem.max()],
[y_test_mem.min(), y_test_mem.max()], 'k--', lw=2)
plt.xlabel('Actual GPU Memory Used (bytes)')
plt.ylabel('Predicted GPU Memory Used (bytes)')
plt.title('Actual vs Predicted GPU Memory Usage')
plt.show()
```

Dataset Used:

This dataset is derived from MIT supercloud's GPU job metrics. It is a detailed log of GPU resource usage per job on an HPC system, recorded via Data Center GPU Manager(DCGM). It contains 96,893 rows (GPU jobs) and 23 features.

The dcgm.csv dataset is a detailed log of GPU job performance metrics collected from an HPC (High Performance Computing) environment, likely derived from the MIT Supercloud infrastructure. Each row in the dataset represents a single GPU job or a single GPU's activity during a job. It contains 96,893 entries and 23 columns, capturing telemetry data from NVIDIA's Data Center GPU Manager (DCGM). The primary purpose of the dataset is to enable the prediction of two key performance indicators: the total execution time of a job (totalexecutiontime_sec, similar to CPUTimeRAW) and the maximum memory usage (maxgpumemoryused_bytes, similar to MaxRSS).

The dataset includes both identifier columns such as Node, gpu_id, and id_job, and numerous numeric features that track GPU utilization, memory usage, power consumption, bandwidth, and energy usage. For example, it records the average and peak memory utilization percentages, streaming multiprocessor (SM) usage, incoming and outgoing PCIe bandwidth, and power consumption at different time intervals. These features reflect the real-time resource usage and are crucial for building regression models to predict resource demands of jobs before they run.

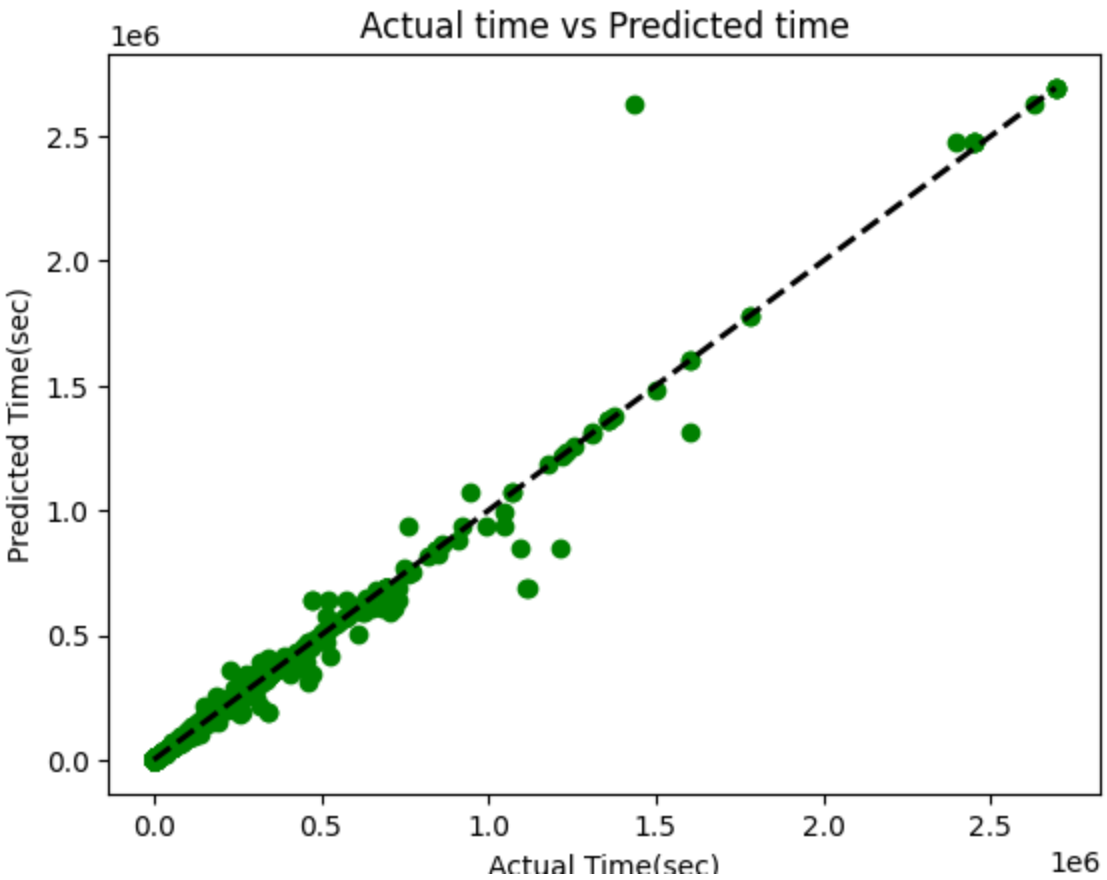
The dataset is GPU-centric rather than CPU-centric like the one used in the original Slurm ensemble prediction paper, it closely aligns in structure and purpose, making it a practical alternative for implementing similar predictive modeling workflows.

How the data was used:

Derived features like power x time, average GPU utilization, and normalized metrics were introduced to improve predictions. For data pre-processing, we have used label-encoding for categorical values in dataset. Categorical values cannot be directly used in regression models.

Output:

GRAPH-1:



Explanation:

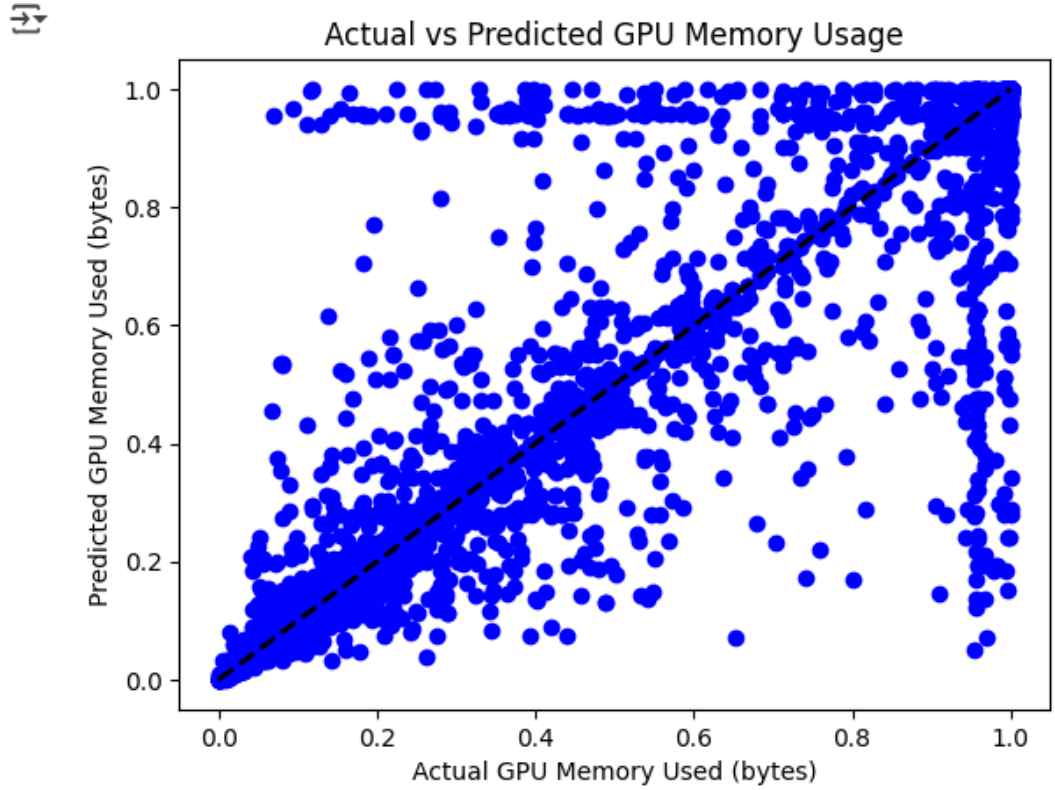
X-axis: Actual execution time of GPU jobs

Y-axis: Predicted execution time by the DTR model.

The dashed black line represents perfect predictions, where predicted execution time exactly equals the actual execution time.

In the following output, the green dots are closely clustered around the diagonal dashed line. This indicates that the model predicted execution time very accurately for most jobs. And only few datapoints are scattered. This indicates that the model we used to predict CPUTimeRAW is accurate

GRAPH-2:


Actual vs Predicted GPU Memory Usage
Predicted GPU Memory Used (bytes)
Actual GPU Memory Used (bytes)
0.0
0.2
0.4
0.6
0.8
1.0

Explanation:

X-axis: Actual maximum GPU memory used

Y-axis: Predicted memory usage by the model.

The dashed black line represents perfect prediction. Points on the line represent accurate predictions.
In this case, the blue dots follow the general diagonal trend, but with more spread compared to the time prediction. This means the prediction is reasonably good but it is not as accurate as the first graph. The lower accuracy in GPU memory prediction stems from missing or noisy features, non-linear and irregular patterns, and model limitations of a single Decision Tree. By enriching the features and using more robust models, accuracy can be significantly improved.

Conclusion:

This project demonstrates the use of decision tree regression(DTR) model in predicting the execution time and physical memory used by different programs. This model was created for implementing the machine learning model that was suggested in the research paper “Ensemble Prediction Slurm”. Decision Trees were chosen for their interpretability and ability to handle both numerical and categorical features without requiring feature scaling.